



Høgskulen
på Vestlandet



DAT108 - Programmering og webapplikasjoner

F06 – Tråder, race conditions og synchronized

Lars-Petter Helland
Bergen
4. september 2026

Plan for i dag

- › En intro til tråder i Java
- › Delte ressurser, race conditions og trådsikkerhet
- › Nøkkelordet synchronized
- › Java minnemodell



Læringsressurser til i dag

Fra Java Concurrency and Multithreading Tutorial (<http://tutorials.jenkov.com/java-concurrency/index.html>):

- › Java Concurrency and Multithreading Tutorial
- › Multithreading Benefits
- › Multithreading Costs
- › Creating and Starting Java Threads
- › Race Conditions and Critical Sections
- › Thread Safety and Shared Resources
- › Thread Safety and Immutability
- › Java Memory Model (første halvdel)
- › Java Synchronized Blocks

+ artikel: <https://www.baeldung.com/java-thread-safety>

The screenshot shows the JENKOV.COM website with the title 'Tech & Media Labs'. The main content area is titled 'Java » Java Concurrency » Java Concurrency and Multithreading Tutorial'. It features a table of contents with the following items:

essentials	Java Language	Java Concurrency and Multithreading Tutorial
tralized ns	Java Collections	Multithreading Benefits
orph	Java Concurrency	Multithreading Costs
formats	Java Concurrency Utils	Concurrency Models
ech		Same-threading
	Java Cryptography	Single-threaded Concurrency
/M		Concurrency vs. Parallelism
ools	Java Date & Time	Creating and Starting Java Threads
fly	Java Exception Handling	Race Conditions and Critical Sections
nitecture	Java Functional Programming	Thread Safety and Shared Resources
are reneurship	Java Generics	Thread Safety and Immutability

On the right side, there is a section titled 'Java Conc' with a list of topics:

- Java Concurr
- What is Multithre
- Why Multithreadi
 - Better Utiliz
 - Better Utiliz
 - Better User
 - Better User
- Multithreading vs
 - Multitasking
 - Multithreadi
- Multithreading is
- Multithreading an
- Concurrency Moc
- Java Concurr

At the bottom right, there is a profile for Jakob Jenkov, Last update: 202', with social media icons for Twitter, LinkedIn, YouTube, and RSS.

Hva er multithreading (fleroppgavekjøring) ?

- › I de Java-programmene dere har laget så langt utføres setningene i `main()` i sekvens til `main()` er ferdig.
- › Programmet går ikke videre til neste setning før forrige setning er utført. Programmet har kun én tråd (main-tråden).
- › Multithreading (en type multitasking) betyr at flere sekvenser av setninger (delprogrammer) kan utføres samtidig i parallell.



Hva er multithreading (fleroppgavekjøring) ?

- › Eksempler:
 - › Et spill der ting beveger seg samtidig som det venter på brukerinput
 - › En webtjener som behandler forespørsler samtidig som den venter på nye
 - › Beregninger av store datamengder der man kan utnytte flere CPU-kjerner
- › Man kan tenke på hver tråd som en "virtuell CPU" der ting utføres uavhengig av de andre trådene.



Hvorfor multithreading?

Noen vanlig grunner:

- › Bedre utnyttelse av en enkelt CPU (f.eks. i fbm. IO)
- › Bedre utnyttelse av flere CPU-er eller kjerner (f.eks. oppdeling av beregninger)
- › Bedre brukeropplevelse mhp. respons (unngå venting, intelligent oppdatering, ...)
- › Bedre brukeropplevelse mhp. rettferdighet (f.eks. på en webtjener)



Multithreading vs. multitasking

- › Multitasking:
 - › CPU-ressurs fordeles mellom ulike program (prosesser).
 - › Operativsystemet bytter frem og tilbake mellom hvem som får bruke CPU (og minne og ...).
- › Multithreading:
 - › Ressursene fordeles mellom ulike tråder **innen ett og samme program**.
 - › Trådene har tilgang til samme minne (objekter). Fordeler og **utfordringer**.



Multithreading og samtidighet i Java

- › Java var et av de første språkene som gjorde multithreading lett tilgjengelig for utviklerne.
- › Det finnes ulike "modeller" for hvordan man kan arbeide med samtidighet i Java.
- › Vi skal i hovedsak se på modellen som kalles "*shared state concurrency model*" der vi jobber med tråder som deler minne (objekter).
- › Nyere modeller, som f.eks. "*shared nothing*" / "*separate state*" har kommet i språket siden. Kanskje vi ser litt på det på slutten av modulen.



Fordeler med multithreading

- › Bedre utnyttelse av CPU
 - › se eksempel i tutorial, innlesing og prosessering av flere filer
- › Enklere programutforming
 - › se eksempel i tutorial, samme som over, kan tenke på færre ting
- › Bedre respons i programmer
 - › se eksempel i tutorial, webtjener venter på og behandler forespørsler
 - › se eksempel i tutorial, desktop app, UI må vente på databehandling
- › Bedre rettferdighet
 - › se eksempel i tutorial, webtjener



Kostnader med multithreading

- › Mer kompleks utforming
 - › Vi må passe på at trådene ikke «går i beina» på hverandre
 - › Vi må passe på at data ikke blir «rotet til» (trådsikkerhet)
 - › Vi må som oftest ha kommunikasjon mellom trådene
 - › Vi må passe på at programmet ikke går i vranglås o.l.
- › Kontekst-switching
 - › Prosessoren må hele tiden bytte på hva den utfører, og samtidig holde orden på alle tråder som venter på tur.
- › Økt ressursforbruk
 - › Et program med mange tråder vil kreve mer både av prosessor og minne



Opprette og starte tråder i Java

IntelliJ-prosjekt i dag: 📁 f06-traader-intro

Pakke for eksempelet: 📦 no.hvl.dat108.f06a

Java-filer:

- 📄 _OppretteTraaderMain
- 📄 MinRunnable
- 📄 MinTraad



Oppsummert fra f06a

- › En tråd er altså en "virtuell" CPU som utfører Java-kode.
- › Vi har alltid minst én tråd, kalt main-tråden.
- › Du kan opprette og starte nye tråder fra programmet som kjører kode i parallell med main-tråden.
- › En tråd i Java er et objekt av typen **java.lang.Thread**.
- › Koden som skal kjøres av tråden plasseres i metoden **run()**.



Opprette tråd-objekter, alternativ 1

Lage en trådklasse som arver fra Thread:

```
public class MinTraad extends Thread {  
    @Override  
    public void run() {  
        ...  
    }  
}
```

```
Thread t = new MinTraad();
```



Opprette tråd-objekter, alternativ 2

Lage en anonym trådklasse som arver fra Thread:

```
Thread t = new Thread() {  
    @Override  
    public void run() {  
        ...  
    }  
}
```



Opprette tråd-objekter, alternativ 3

Lage en klasse som implementerer Runnable-kontrakten, og som brukes i konstruktøren til Thread ved opprettelse av tråd-objekt:

```
public class MinRunnable implements Runnable {  
    @Override  
    public void run() {  
        ...  
    }  
}
```

```
Thread t = new Thread(new MinRunnable());
```



Opprette tråd-objekter, alternativ 4

Lage en anonym klasse som implementerer Runnable-kontrakten, og som brukes direkte i konstruktøren til Thread:

```
Thread t = new Thread(new Runnable() {  
    @Override  
    public void run() {  
        ...  
    }  
});
```



Opprette tråd-objekter, alternativ 5

Siden `Runnable` er et `@FunctionalInterface` ved at det kun har én metode, `void run()`, kan dette representeres som et lambda-uttrykk:

```
Thread t = new Thread(() -> { ... });
```



Starte en tråd

```
Thread t = ...
```

```
t.start();
```

Hva skjer om vi skriver

```
t.run();
```



Å identifisere tråder

- › Tråder kan gis navn:

```
... = new Thread("Tråd1");
```

```
... = new Thread(myRunnable, "Tråd2");
```

Hvis `run()` er definert i en subklasse av **Thread** kan man skrive:

```
... getName(); i run()-metoden.
```

Hvis `run()` er definert i en **Runnable** som gis i konstruktøren til **Thread** må man skrive:

```
... Thread.currentThread().getName(); i run()-metoden.
```



Tråder med navn

IntelliJ-prosjekt i dag: 📁 f06-traader-intro

Pakke for eksempelet: 📦 no.hvl.dat108.f06b

Java-filer: 📄 _TraaderMedNavnMain



Å pause en tråd. Å stoppe en tråd

- › Ønsker vi at gjeldende tråd skal ta en **pause** i utførelsen bruker vi sleep:
`Thread.sleep(millis) ;`
- › Hva med å **stoppe** en tråd? Ofte vil run-metoden ha en "uendelig løkke" som utfører det samme om og om igjen til vi ønsker å avslutte.
- › Vi trenger en måte å gi beskjed til tråden at vi ønsker å bryte av. Vanligvis gjøres dette med en boolsk variabel som vi sjekker på i den "uendelige løkken".
- › **Merk!** Thread har en (deprecated) stop()-metode. Ikke bruk denne.



Å vente (eller ikke) på at en tråd avsluttes

- › Hvis vi f.eks. i `main()` har opprettet og startet noen tråder, hvordan kan vi be `main()` å **vente** på at tråder gjør seg ferdige før `main()` fortsetter?
- › Når avsluttes programmet? Jo, når ALLE (ikke-deamon-) tråder er ferdige.
- › Hvordan kan vi sette opp slik at programmet avsluttet FØR alle tråder er ferdige?



Pause, stoppe, vente, daemon

IntelliJ-prosjekt i dag: 📁 f06-traader-intro

Pakke for eksempelet: 📦 no.hvl.dat108.f06c

Java-filer: 📄 _SleepStoppJoinMain

📄 TraadSomKanStoppes





Delte ressurser og "kappløp"

- › Ofte vil flere tråder jobbe med de samme dataene.
- › Det kan oppstå konflikter hvis flere tråder prøver å oppdatere data samtidig.
- › Hvis vi har en bit av koden som jobber på delte data (f.eks. oppdaterer data), og det er viktig at disse kodelinjene utføres isolert (der kun én tråd har tilgang om gangen), kalles denne seksjonen for en **kritisk seksjon**.
- › En kappløpssituasjon (**race condition**) er en uønsket situasjon der flere tråder har tilgang til en kritisk seksjon samtidig, og at rekkefølgen på utførelsen påvirker resultatet.
- › **Vi må altså beskytte kritiske seksjoner mot race conditions.**



[Threads in a Tangle](#)

TelleOpp og TelleNed - Problemet

IntelliJ-prosjekt i dag: 📁 f06-traader-intro

Pakke for eksempelet: 📦 no.hvl.dat108.f06d

Java-filer: 📄 _RaceConditionMain

📄 TelleNedTraad

📄 TelleOppTraad

📄 Teller

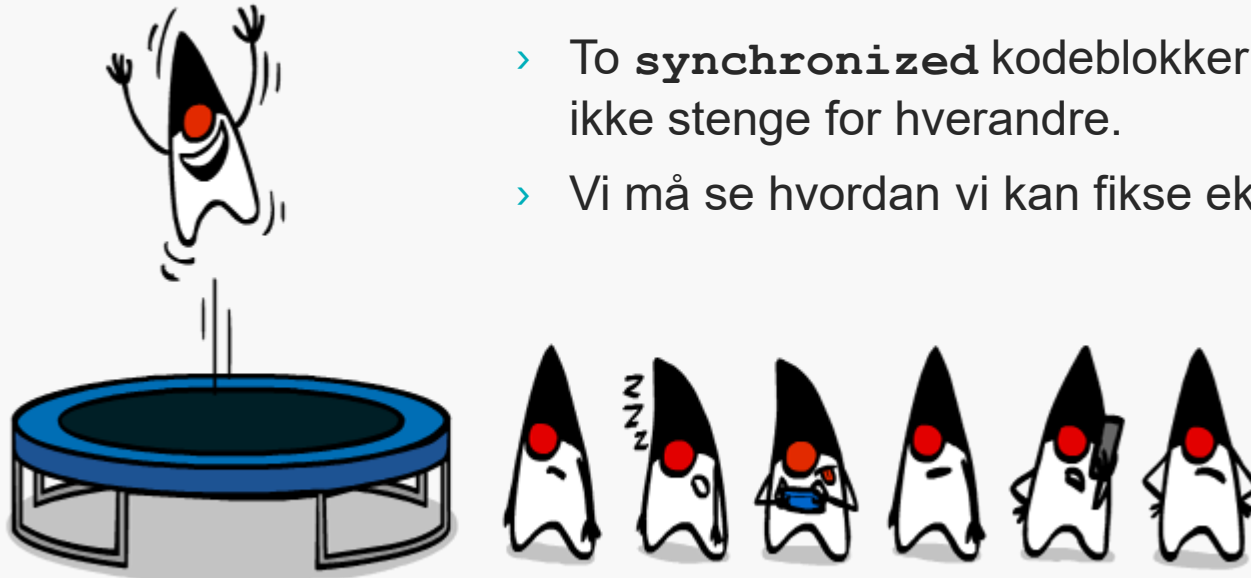


Så hvordan løser vi dette?



synchronized

- › En **synchronized** kodeblokk kan kun utføres av én tråd om gangen.
- › Når vi bruker **synchronized** kan vi tenke på det som at kun den tråden som får "ballen" får utføre koden i blokken. De andre trådene må vente. Når tråden er ferdig med blokken kan "ballen" gis videre til neste tråd.
- › "Ballen" kan være hvilket som helst Java-objekt.
- › To **synchronized** kodeblokker kan bruke ulike "baller" og dermed ikke stenge for hverandre.
- › Vi må se hvordan vi kan fikse eksempelet vårt med **synchronized**



TelleOpp og TelleNed - Løsningen

IntelliJ-prosjekt i dag: 📁 f06-traader-intro

Pakke for eksempelet: 📦 no.hvl.dat108.f06d

Java-filer: 📄 _RaceConditionMain

📄 TelleNedTraad

📄 TelleOppTraad

📄 Teller



Oppsummert fra eksemplet

- › Den kritiske seksjonen er koden i de to trådene som endrer verdi på telleren
- › Vi kan bruke teller-objektet som "ball" og merke kodeblokkene med

```
synchronized (teller) {  
    teller.<kode som endrer på verdien i teller>  
}
```

- › Vi kan alternativt gjøre teller **trådsikker** ved å markere kodeblokk eller oppdateringsmetodene (opp og ned) i teller slik

```
public synchronized void opp() {  
    verdi++;  
}
```

- › I en **synchronized** metode er det objektet som eier metoden som blir "ballen", altså **this**.



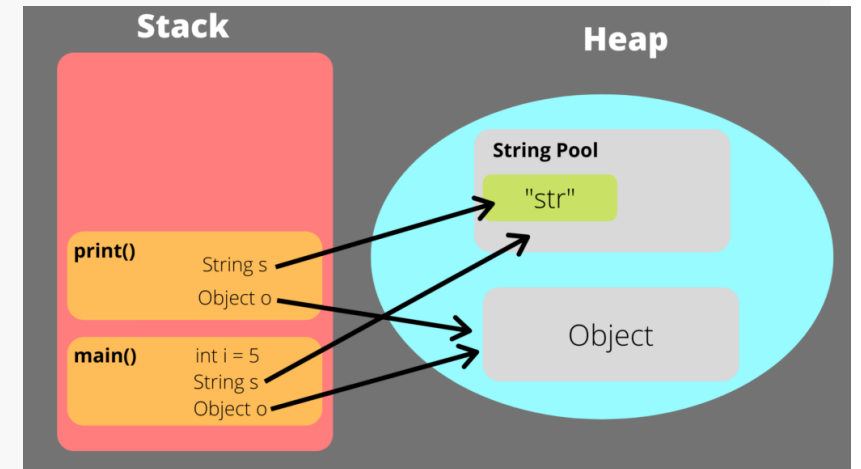
Ballen

- › Når man skriver `synchronized(ballen)` så brukes en spesiell egenskap ved objektet `ballen`.
- › Alle objekter i Java er/har en såkalt **monitor** / monitorlås / innebygget lås. Det er denne mekanismen i objektet som brukes, og som holder orden på hvilken tråd som "har låsen", og hvilke som venter, osv...
- › Dere lærer mer om begrepsbruk i DAT103.

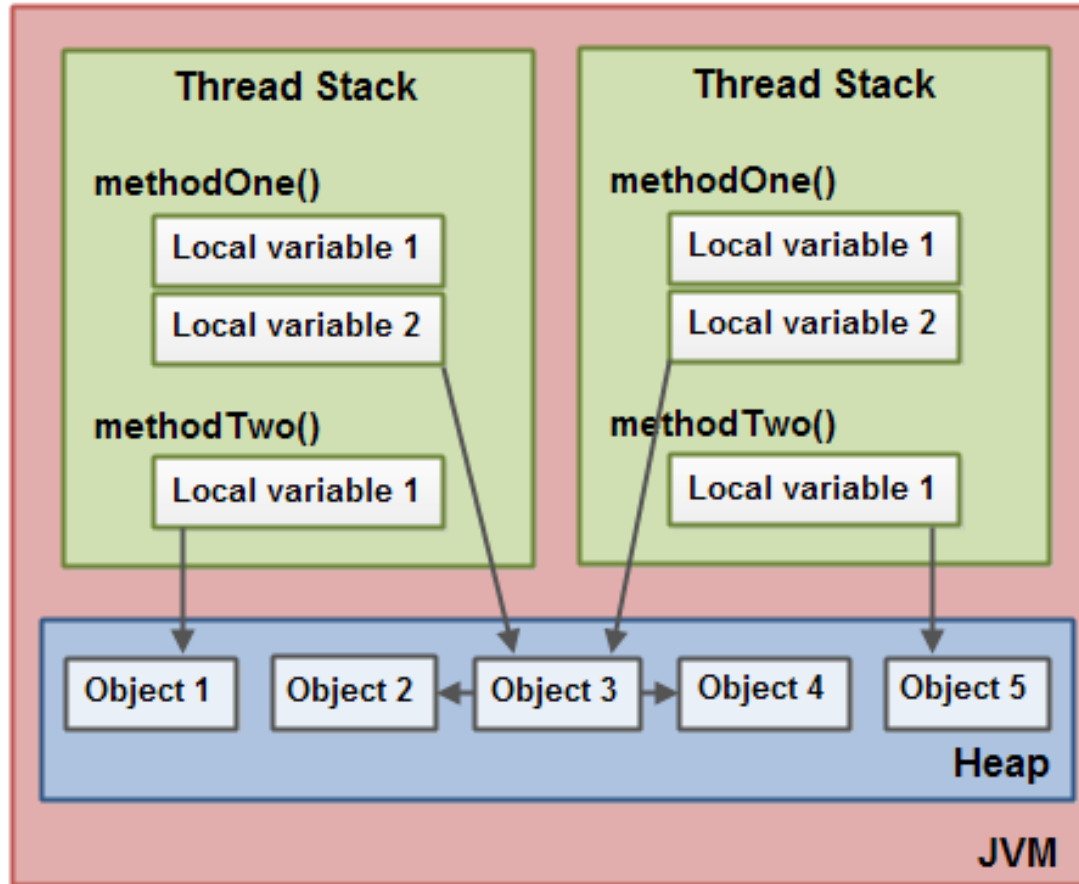


Trådsikkerhet og delte ressurser

- › Kode som trygt kan kalles av flere tråder samtidig kalles **trådsikker**.
- › Trådsikker kode inneholder ingen **race conditions**.
- › Lokale variabler (parametre og variabler i metoder):
 - › Vi så litt på dette i fbm. rekursjon i DAT102 ... Lokale variabler lagres i hver tråd sin egen **stack**. De deles ikke mellom tråder, og er automatisk trådsikre.
- › Lokale objektreferanser:
 - › Selv om variabelen/referansen er lokal (og lagres på stack-en), vil selve objektet som det refereres til lagres en i felles **heap**. Her må man passe litt på.
- › Objekters medlemsvariabler:
 - › Et object sine object-/instansvariabler lagres på **heap**-en sammen med objektet.

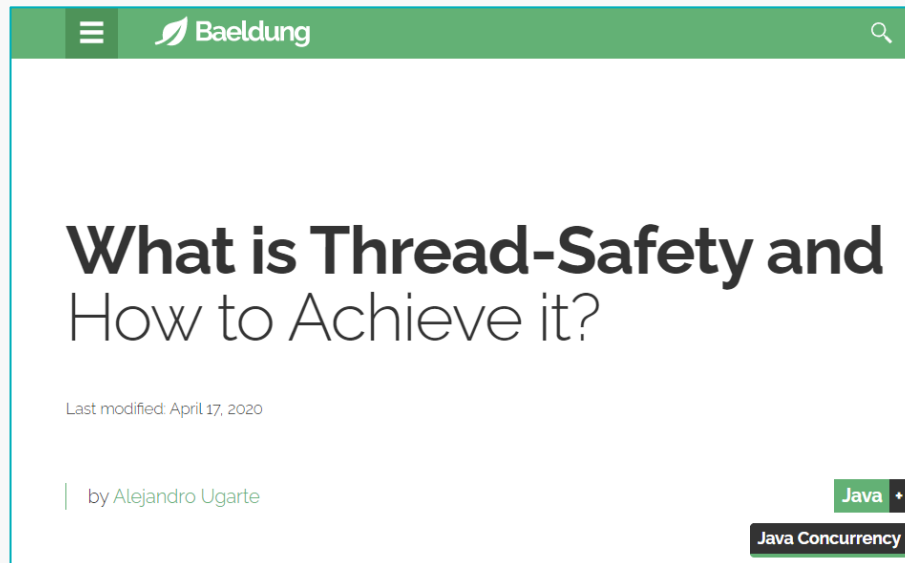


Java Memory Model



En grei oversiktsartikkel om trådsikkerhet

- › <https://www.baeldung.com/java-thread-safety>
m/kodeeksempler på
<https://github.com/eugenp/tutorials/tree/master/core-java-modules/core-java-concurrency-basic>
- › Jeg foreslår at dere leser litt på den til neste uke, så kan vi se på et par eksempler.



Videre

Oblig1: Inn søndag

Neste uke:

- › Sende signaler mellom tråder. wait(), notify(), notifyAll(). Producer - Consumer

Oblig2:

- › Legges ut neste uke
- › **NB! Kort frist søndag 20. sep.**

Lab neste uke:

- › Jobbe med Oblig2 så langt dere klarer

